Supplementary material for CON 3.1

Rohit Goswami[a,1]

[a] *TurtleTech ehf., Reykjavík, Iceland*

Abstract

These notes record the measurements behind the CON 3.1 paper. The restart displacements, the format matrix, the frame bytes, the velocity step and the illegal-file classes are reads of the campaign files and the conformance corpus. The language-pair table and the MPI rank tables are the cloud campaign. The ingest tables are the readcon-db evaluation at commit `6146c3b`.

S1. Restart difference

The interrupted band in Section 6 writes a checkpoint and then continues. Figure S1 compares each resumed movie with the uninterrupted movie at the same index. Column 3 is that checkpoint. The final column is the band after the five steepest-descent steps.

S2. Round-trip fidelity

Figure S2 uses two unmodified files. The matrix rows other than the velocity-file row are the first frame of the height-held EMT band. That frame already carries mask 4, a z-only hold, on the adatom, together with ids, forces and an energy. It carries no velocities and no time unit, so the time-unit cell on those rows is absent. The velocity-file row is a separate round trip of the unmodified 28-atom velocity fixture, whose time unit is the natural unit. Neither file was edited. The library conversion into ASE atoms passes energy and forces as positional arguments. ASE 3.29.0 accepts those values only as keywords, so the measurement supplies them as the keywords that constructor requires. The grades are the fields that come back from a write and a read after that conversion.

Email address: `rgoswami@ieee.org` (Rohit Goswami)
[1]ORCID: 0000-0002-2393-8056.

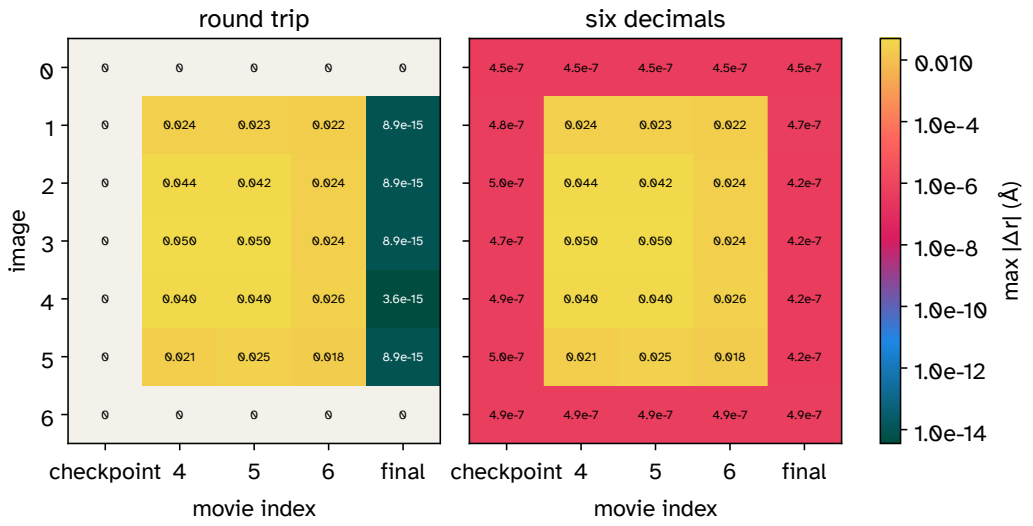


Figure S1: Minimum-image maximum component of the displacement, in Å, between the uninterrupted Morse band and each resume. The checkpoint column is bitwise exact for the round-trip printer and about 5×10^{-7} Å for six decimals. The final column is at most 8.9×10^{-15} Å for the round trip and 4.9×10^{-7} Å for six decimals. Intermediate columns differ by up to 0.050 Å because the search has taken further steps after the load.

S3. Frame bytes and parse time

The frame is the 28-atom velocity file, 3,826 bytes. The timed files are 400 copies of that frame and 400 copies with the velocity section removed. The reported times are medians of 31 paired reads.

S4. Velocity mislabel

The kinetic energies and the Verlet step are the illustration in Section 6. The conversion from the natural unit to femtoseconds is the factor used for that illustration. The cell is the slab cell of the file.

S5. Illegal-file classes

The eight illegal files are the conformance corpus named in Section 6. readcon raises a message that names the class. ASE 3.29.0 reads the same files with its eon reader.

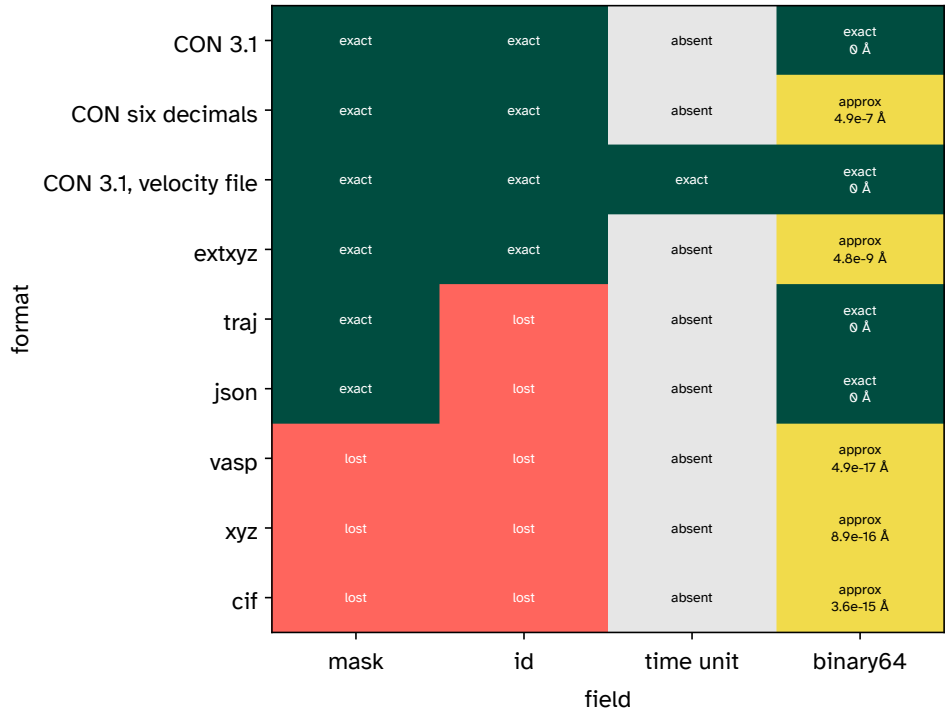


Figure S2: Fields kept by a write and a read. The held-frame rows have no time unit in the source file. The velocity-file row keeps the natural unit. CON 3.1 keeps the mask, the id and binary64 on both files. Six decimals keep the mask and the id and move a held-frame coordinate by 4.9×10^{-7} Å. Extended XYZ keeps the mask and the id and moves a coordinate by 4.8×10^{-9} Å. The trajectory and JSON writers keep the mask and the coordinates and drop the id. POSCAR, XYZ and CIF drop the mask and the id. The number in the binary64 cell is the maximum absolute coordinate change. A grey cell is a time unit the source frame did not carry.

S6. Database hashes of the campaign bands

The hashes are a query against readcon-db 0.1.8. The five bands were appended as stored trajectories, and each frame hash was read back by the same query. The bands are the uninterrupted Morse restart, its round-trip resume, its six-decimal resume, and the two EMT mask bands of Section 6. The query returns 35 images and 35 hashes. Every returned hash locates the frame that was stored. Endpoint coordinates of the uninterrupted band and the round-trip resume agree: the maximum absolute component is 0 at both endpoints and 8.9×10^{-15} Å over the seven images. The stored texts still differ, so the hash keeps every frame apart. On the reactant the only

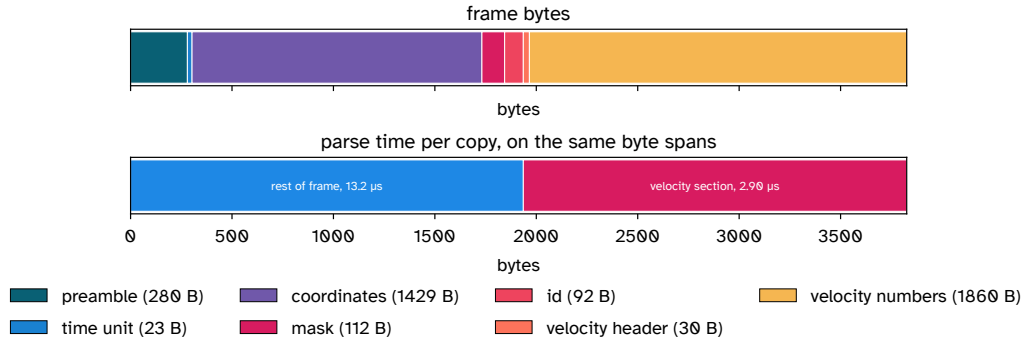


Figure S3: Byte widths of one CON 3.1 frame, and the parse time of the same spans. Masks, ids and the time unit are 227 bytes, against 1,429 coordinate bytes and 1,860 velocity-number bytes. The velocity section adds 2.90 μs to a 13.2 μs read of the rest of the frame. Masks and ids share the coordinate line, so their parse time is inside that 13.2 μs.

differing line is the metadata line. The printed curvature products differ by one byte, 2,507 bytes against 2,506, and the coordinate rows match. The product frame differs only in that same metadata line. The five intermediate images differ in the metadata line and in coordinate rows at the last printed digits, which is the 8.9×10^{-15} Å displacement. Figure S7 is the same five bands on the (s, d) plane [1].

S7. Failure cases: frame, grading and drift files

The survey frame is a Pt(100) $2 \times 2 \times 2$ slab with 6 Å of vacuum and one Pt adatom 1.8 Å above the hollow site. Every coordinate carries a uniform offset of up to 10^{-3} Å from a fixed seed, so all seventeen significant digits are needed. The bottom four atoms are fixed on every axis, and the adatom is fixed in z. The ids are 17, 3, 11, 5, 23, 2, 13, 7 and 41 in row order. Velocities are normal deviates of standard deviation 4×10^{-3} in the natural unit, zero on held axes. For ASE the masks become one FixAtoms and one FixCartesian, the ids an integer array named `id`, and the velocities ASE momenta. For MDAnalysis the velocities are converted to Å/ps, its base unit. We count a field as kept when it reads back equal; a mask counts as a full freeze when every masked atom returns frozen on every axis. A velocity is declared when the file names its unit and the values agree after conversion to within 10^{-6} relative. When the file names no unit, we report what a reader that takes the numbers as Å/fs would see, a kinetic energy 103.64 times too large.

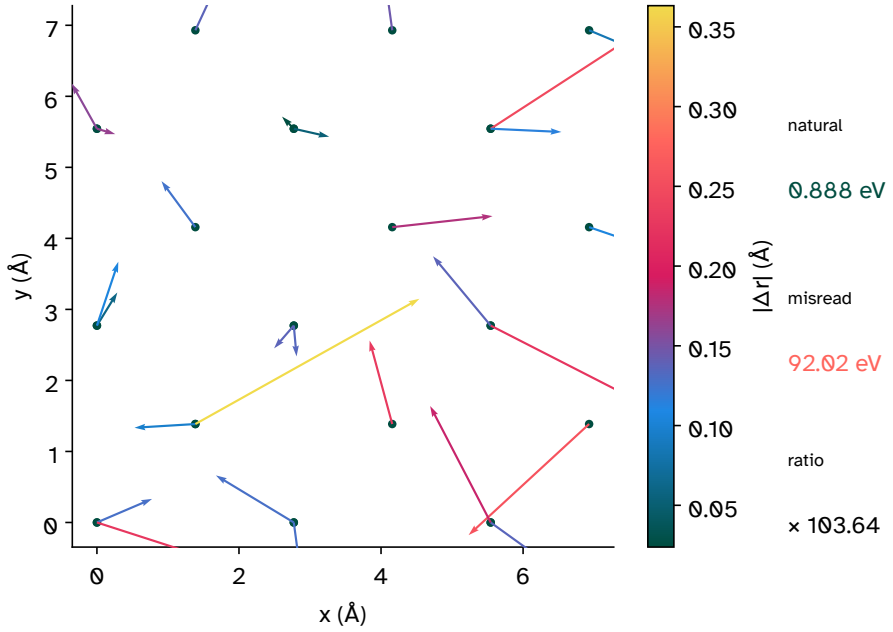


Figure S4: Displacement after one velocity Verlet step of 1 fs on the 28-atom slab when the natural velocity is misread as Å/fs. The kinetic energy goes from 0.888 eV to 92.02 eV, a factor of 103.64. The RMSD is 0.096 Å and the maximum component is 0.315 Å. The arrows are drawn at 10 times that displacement.

Each drift file has three Pt atoms with the header

```
Lattice="5.5437171645025325 0.0 0.0 0.0 5.5437171645025325 0.0 0.0 0.0
    20.0" Properties=species:S:1:pos:R:3:move_mask:L:3:id:I:1 pbc="T T F"
```

and rows `Pt 0.0 0.0 6.0 T T T 17`, `Pt 2.7718585822512662 0.0 6.0 T T T 3` and `Pt 1.3859292911256331 1.3859292911256331 7.8 F F T 41`. The variants drop the `pbc` key, replace the header by a plain comment and the rows by their first four fields, write the z value 6.0 as `6.0d0`, spell the booleans `True` and `False`, add `note="E=1.5 eV"`, and add `units="angstrom,eV,fs"`. The readers are ASE 3.29.0, the libAtoms extended XYZ parser 0.4.6 through its C path and its Python path, chemfiles 0.10.4, and MDAnalysis 2.10.0. The correctly rounded binary64 value of 2.7718585822512662 has the bit pattern `40062cc4313f3fb9`; chemfiles returns `40062cc4313f3fb8`.

The resume study uses an EMT Pt(100) $3 \times 3 \times 3$ slab with 6 Å of vacuum, the bottom layer held, and one adatom 1.75 Å above a hollow and held in z, hopping to the neighbouring hollow. Endpoints were relaxed with BFGS

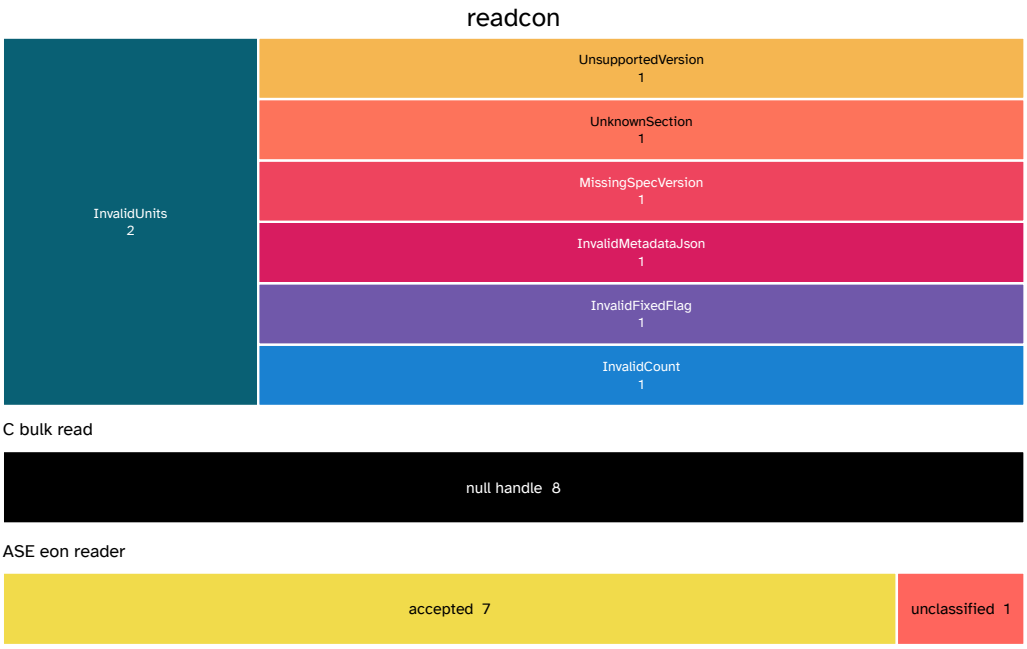


Figure S5: Classes on the eight illegal files. readcon names a class for each file. The ASE eon reader accepts seven of the files and stops without a class on the per-type count of zero.

to 0.002 eV/Å. Five movable images were interpolated with the constraints applied. The band uses the improved tangent with a climbing image, and each step adds 0.02 Å^2/eV times the projected force. We interrupt after step 25. Each route writes all seven images and reads them back with the reader the route names, keeping whatever constraints and periodicity that reader returns.

S8. Reader rules

Table S1 collects the edge cases a reader applies. They are the classes of Section 3, stated as input and outcome.

S9. Paired peer timings on every payload

Table S2 extends the peer table of the main text to the 1-frame and 100-frame files. The comparison calls the Python binding with one worker,

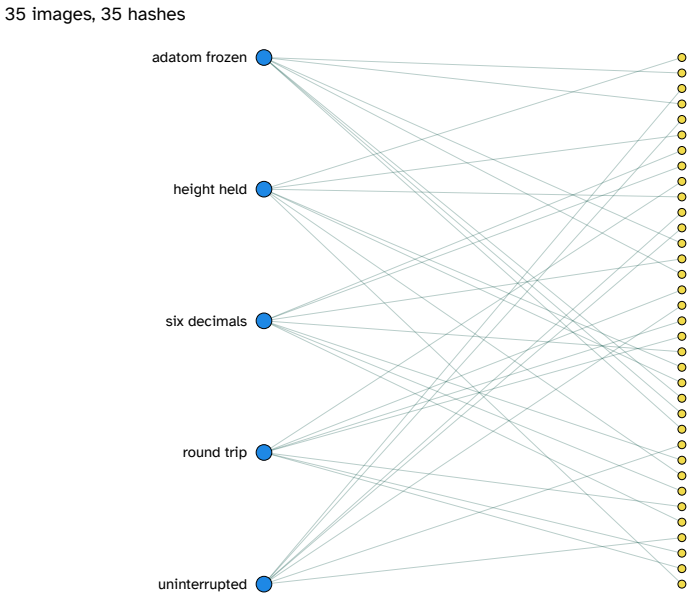


Figure S6: Campaign bands on the left and the database hash of each stored frame on the right. The query returns 35 images and 35 hashes. No hash is shared. The endpoints match in coordinates. Their hashes differ because the metadata line does.

and ASE, chemfiles, MDAnalysis and h5py run in the same process with OpenMP and BLAS limited to one thread. XYZ and H5MD peers receive the coordinates of the same frames, written before the timed loop starts. Each cell is one-worker readcon-core, then the peer, thirty times, after one untimed call of each side. A repeat over 240 s would be recorded as a timeout and a peer that raises as a failure; neither occurred. The interval is a bias-corrected and accelerated bootstrap of the median with 9999 draws [2]. On one frame the fixed cost of opening the file dominates, which is why h5py and chemfiles lose their lead there.

S10. Stored ingest timings

readcon-db answers composition, energy and maximum-force queries over CON frames. The numbers are the evaluation at commit `6146c3b`: five interleaved repeats, median then minimum and maximum. The machine was an Intel Xeon with 8 CPUs and 47.07 GiB of memory. The software was Python 3.12.3, rustc 1.88.0, readcon-db 0.1.8, readcon-core 0.16.2, ASE 3.29.0

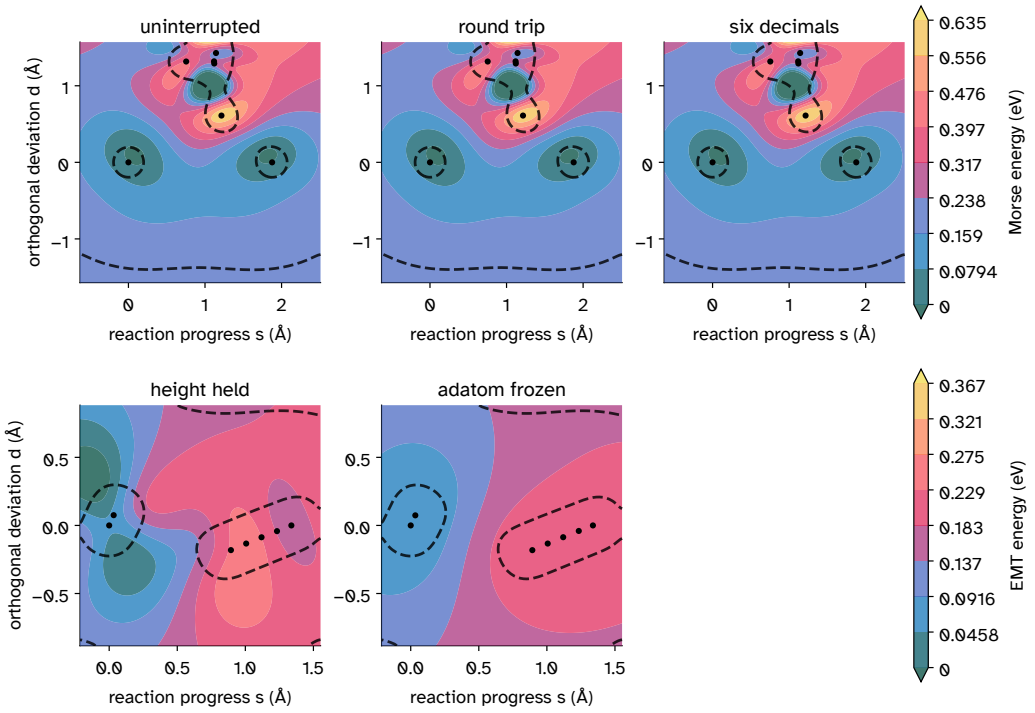


Figure S7: The same five bands on the (s, d) plane [1]. The Morse maps share one colour scale and the EMT maps share another. Black dots are the images. The round-trip and six-decimal maps sit on the uninterrupted Morse band at this scale.

and numpy 2.5.4. Threads were pinned to one so that readcon-db, ASE's database and the CON directory ran under the same cap. The parallel ingest is the MPI campaign in Section S12. A megabyte is 10^6 bytes. A cold query follows `posix_fadvise` with `DONTNEED`, then runs in a fresh process. The run did not drop the page cache. A warm query is the next query in that same process. The timed corpora repeat ASE effective-medium searches: a hydrogen cyanide band, a copper adatom band, a platinum-hydrogen band and a platinum adatom dimer. Every frame in those corpora has its own xxHash3-128, at both 1,000 and 10,000 frames, so the ingest timing is the cost of storing distinct texts. The clock starts before the write and stops after a composition check. The ingest table has 30 rows and the query table has 180, and every row matched the hit count and the sum of frame indexes. Python `ConCorpus.select` always applies an atom-count range, including the default from 0 to $2^{32} - 1$, so a Python query walks that index. The timed queries

Table S1: Reader rules for flags, versions, identity and incomplete input.

Input	Rule
Column 4 outside the legal set	Illegal flag.
Version 1 column 4 equal to 1	Fixed on x, y and z.
Version 1 column 4 other than 0 or 1	Illegal flag.
Missing id column	Id is the zero-based row index.
Repeated id	File order is kept.
Unique ids	Ascending id is the pre-grouping order.
Unknown major	Rejected.
Unknown minor of a known major	Accepted.
Unknown metadata key	Kept on rewrite.
Unknown section at this minor	Rejected.
Unknown section at a newer minor, width declared	May be skipped.
Non-finite number	Rejected.
Type count of zero	Legal empty frame.
Short header or short section	No partial frame is returned.
Empty first header line	User line, kept.
Empty second header line	Implicit version 1.
Blank line after a frame, no velocity block	Line 1 of the next frame.
Several gzip members	Read in order.

use the Rust select and do not add the predicate. `readcon.iter_con` skips a leading newline, so an empty user line shifts the header. The evaluation writes the user line `cpc-eval`. `read_con` on one frame keeps an empty user line, which the grammar allows.

Table S3 is the ingest of 10,000 frames. Inside a single ASE transaction the writer still commits every 5,000 rows, so 1,000 frames is one commit and 10,000 frames is two commits, plus the commit that closes the block. The other ASE column commits on every write. At 1,000 frames the medians are 0.0329 s (30,400 frames/s) for readcon-db, 0.646 s (1,550 frames/s) for one transaction, and 5.03 s (199 frames/s) for a commit on every write. At 10,000 frames readcon-db writes at 35.1 MB/s (33.0 to 38.0), the single transaction at 1.83 MB/s, and the per-write commits at 0.226 MB/s.

On composition, energy and maximum force at 10,000 frames, readcon-db takes about 8 to 10 ms when cold and about 0.2 ms when warm. The cold

Table S2: Paired time ratios. The ratio is the readcon-core time divided by the peer time, so a value below 1 means readcon-core finished sooner. The interval is a bias-corrected and accelerated 95 percent interval of the median. Failures stay in the table and are not converted into times.

Payload	Peer	OK	Fail	Ratio	Interval
1 frame	ASE eon	30	0	0.153	[0.15, 0.155]
1 frame	chemfiles XYZ	30	0	0.893	[0.888, 0.902]
1 frame	h5py	30	0	0.365	[0.353, 0.37]
1 frame	MDAnalysis H5MD	30	0	0.0207	[0.02, 0.0223]
100 frames	ASE eon	30	0	0.235	[0.226, 0.238]
100 frames	chemfiles XYZ	30	0	1.29	[1.26, 1.32]
100 frames	h5py	30	0	14.1	[13.6, 14.8]
100 frames	MDAnalysis H5MD	30	0	0.134	[0.127, 0.139]
10000 frames	ASE eon	30	0	0.337	[0.335, 0.345]
10000 frames	chemfiles XYZ	30	0	2.15	[2.14, 2.17]
10000 frames	h5py	30	0	67.1	[66.5, 67.3]
10000 frames	MDAnalysis H5MD	30	0	0.207	[0.204, 0.213]
Pt slab	ASE eon	30	0	0.233	[0.231, 0.241]
Pt slab	chemfiles XYZ	30	0	1.14	[1.13, 1.16]
Pt slab	h5py	30	0	6.05	[5.91, 6.27]
Pt slab	MDAnalysis H5MD	30	0	0.394	[0.387, 0.404]

medians are 8.14 ms (5.12 to 10.7), 10.1 ms (8.11 to 14.4) and 9.57 ms (7.70 to 11.6), in that query order. The warm medians are 0.176 ms, 0.211 ms and 0.208 ms. The ASE database takes 0.286 s, 0.223 s and 0.225 s on the cold queries, and 0.258 s, 0.203 s and 0.206 s when warm. A directory of the same CON text, in 64-frame chunks, takes 0.193 s, 0.193 s and 0.197 s cold, and 0.140 s, 0.138 s and 0.138 s warm. Figure S10 shows both corpus sizes.

readcon-db occupies 2.50 times the raw CON text, at both 1,000 and 10,000 frames. The ASE database is smaller than the raw text. Figure S11 also shows gzip and zstd of the same text.

One adaptive kinetic Monte Carlo trace of 24,000 frames stores 27.9 MB of blobs, 27,864,917 bytes, equal to the raw file, because the stored texts are kept separately. The hash index of that stored trace records 7 unique structures, and their text is 8319 bytes. Those seven texts are the unhopped hydrogen cyanide state and its saddle, the unhopped copper state and its saddle, and both platinum-hydrogen states together with that saddle. Figure S12 compares the full trace with that unique set. That collapse is a property of the repeated states in the trace. It is separate from the campaign-band

Table S3: Ingest of 10,000 frames from the stored measurements. Each cell is the median, then the minimum and the maximum of five interleaved repeats.

Writer	Time (s)	frames/s
readcon-db	0.311 (0.287 to 0.331)	32,100 (30,200 to 34,800)
ASE, one transaction	5.99 (5.88 to 6.29)	1,670 (1,590 to 1,700)
ASE, commit per write	48.4 (45.5 to 50.7)	207 (197 to 220)

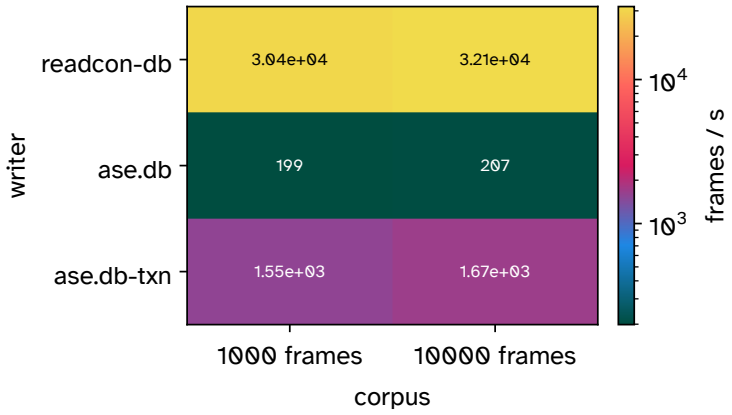


Figure S8: Median ingest rate from the stored measurements. Rows are readcon-db, an ASE database that commits on every write, and one ASE transaction. Columns are the two corpus sizes.

query in Section S6, which found no shared hash.

Nine correctness checks pass on that store. They cover a round trip, identical bytes sharing one hash entry, a one-digit coordinate change, a missing force or velocity section, periodicity taken only from the declared mask, a rebuild of the indexes from the stored text, a second rebuild, an ignored numeric cache, and a hash entry that remains after a duplicate is rewritten. In readcon-db 0.1.8 the LMDB map defaults to 2 GiB (`DEFAULT_MAP_SIZE`). `open_with_map_size` and `READCON_DB_MAP_SIZE` raise it, and a full map grows and retries. The tables in this note were collected at commit `6146c3b`, where the map was that 2 GiB constant, so one million frames returned `MDB_MAP_FULL`. A 100,000-frame ingest was left out of the timed table, because the ASE database at about 200 frames/s exceeds two minutes. gzip and zstd in Figure S11 are wrappers of the CON text. The store keeps each text, so the stored blob bytes of the 24,000-frame trace equal the raw file, and the

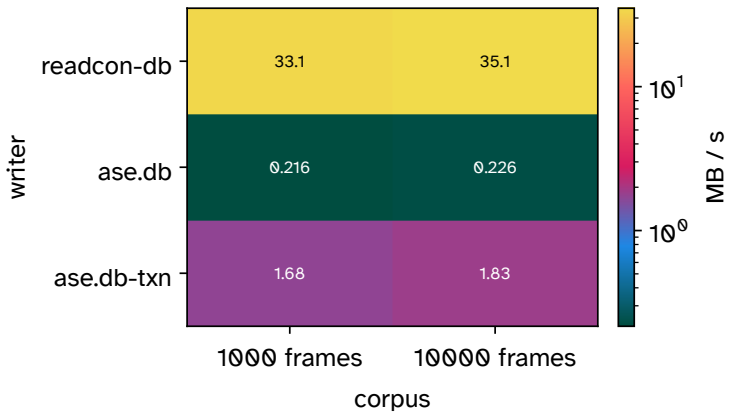


Figure S9: Median ingest bandwidth for the same three writers and the same five repeats. A megabyte is 10^6 bytes.

indexes sit beside that text.

S11. Language pairs

Table S4 is the climbing-image band through each writer and reader among Python, C, C++, Fortran and Julia. The Fortran rows call `rkr_frame_add_atom_full` in the C header order. The public Fortran `add_atom` wrapper binds a different argument order, returns status −4, and is not one of these rows.

S12. MPI ranks

Tables 15 and 17 in the main text are the warm medians. The tables here list every rank that was run, cold and warm. Ranks 16 and 32 were not run. Every repeat matched the serial bytes.

S13. ABNF of one frame

The non-normative ABNF of one frame follows. An empty first header line is the user line, an empty second header line is implicit version 1, and a blank line after a frame is the next frame unless it begins a velocity block. The listing has 50 rules. The specification wins where the two disagree.

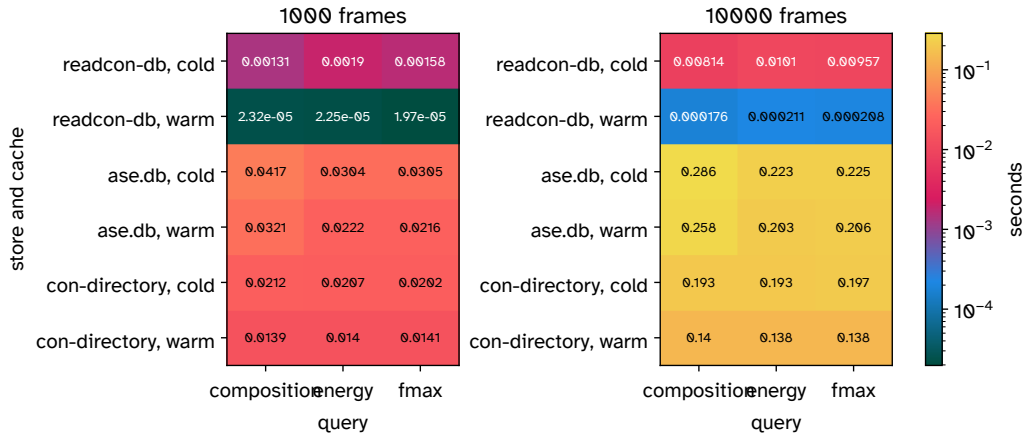


Figure S10: Median query time for composition, energy and maximum force, from the stored measurements. Each panel is one corpus size. Rows pair the store with a cold or warm cache.

Listing S1: Non-normative ABNF for one frame.

```
file = [bom] 1*frame *(*ws nl)
bom = %xFEFF
; An empty free_line on line 1 is the user line.
header_open = free_line nl metadata_line nl float3_line float3_line
    free_line nl free_line nl
frame = header_open (zero_atom / typed_body)
free_line = *(%x00-09 / %x0B-0C / %x0E-FF)
metadata_line = *ws (json_object / legacy_metadata)
; An empty legacy_metadata line is implicit version 1.
legacy_metadata = *ws [(%x00-09 / %x0B-0C / %x0E-7A / %x7C-FF) *(%x00-09 /
     %x0B-0C / %x0E-FF)]
json_object = "{" *ws [json_pair *(*ws "," *ws json_pair)] *ws "}" *ws
json_pair = json_string *ws ":" *ws json_value
json_value = json_string / json_number / json_object / json_array / "true"
     / "false" / "null"
json_array = "[" *ws [json_value *(*ws "," *ws json_value)] *ws "]"
json_string = DQUOTE *json_char DQUOTE
json_char = "\" DQUOTE / "\\" / "\/" / "\b" / "\f" / "\n" / "\r" / "\t" /
    ("\u" 4HEXDIG) / (%x20-21 / %x23-5B / %x5D-FF)
json_number = ["-"] 1*DIGIT ["." 1*DIGIT] [("e" / "E") ["+" / "-"] 1*DIGIT
    ]
float3_line = *ws float 1*ws float 1*ws float *ws nl
count_line = *ws positive_integer *ws nl
count_list_line = *ws positive_integer *(1*ws positive_integer) *ws nl
float_list_line = *ws float *(1*ws float) *ws nl
```

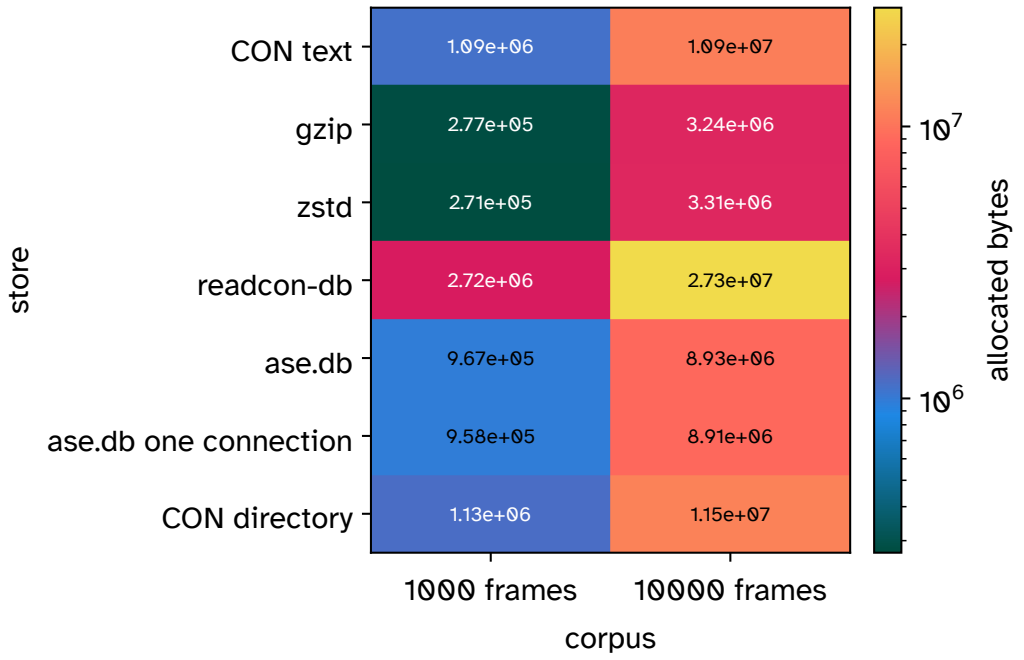


Figure S11: Allocated bytes for the raw text, two compressed copies, readcon-db, both ASE database files and a directory of 64-frame chunks. The readcon-db cell is the memory-mapped file, text plus indexes.

```
zero_integer = 1*"0"
zero_count_line = *ws zero_integer *ws [nl]
blank_line = *ws nl
zero_atom = zero_count_line (blank_line (blank_line / "") / "")
typed_body = count_line count_list_line float_list_line 1*type_block *
    section
type_block = symbol_line nl coord_label nl vector_atom_line *(nl
    vector_atom_line) [nl]
symbol_line = *ws symbol *ws
symbol = ALPHA *(ALPHA / DIGIT / "_")
coord_label = *ws "Coordinates of Component" 1*ws positive_integer *ws
vector_atom_line = *ws float 1*ws float 1*ws float 1*ws integer [1*ws
    integer] *ws
scalar_atom_line = *ws float 1*ws integer 1*ws integer *ws
atom_line = vector_atom_line
; A blank_line is a section only when a block follows, including a
    velocity block.
; Otherwise that line is line 1 of the next frame.
```

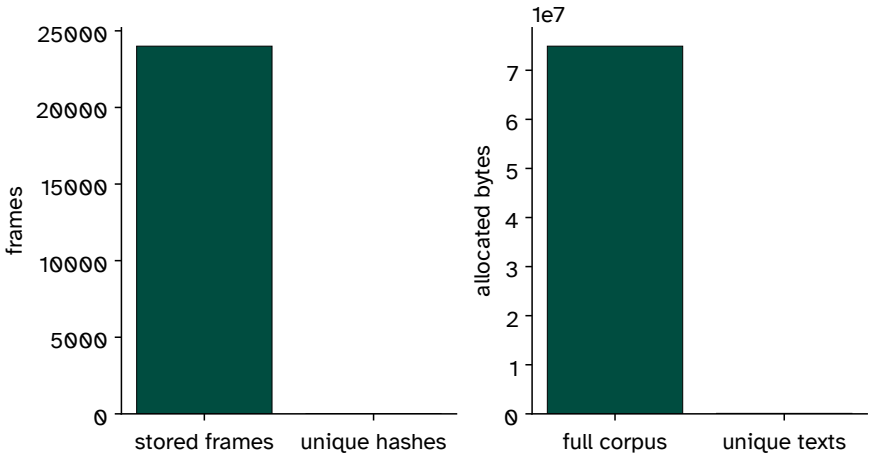


Figure S12: The stored 24,000-frame trace against its 7 unique structures. The left panel counts frames. The right panel is allocated bytes for the full corpus and for a corpus of the unique texts.

```
section = blank_line (1*section_type_block_vector / 1*
    section_type_block_scalar / 1*generic_type_block)
section_type_block_vector = symbol_line nl section_label_vector nl
    vector_section_line *(nl vector_section_line) [nl]
section_type_block_scalar = symbol_line nl section_label_scalar nl
    scalar_atom_line *(nl scalar_atom_line) [nl]
generic_type_block = symbol_line nl generic_label nl token_line *(nl
    token_line) [nl]
generic_label = *ws symbol *(1*ws (symbol / positive_integer)) *ws
token_line = *ws (float / integer) *(1*ws (float / integer)) *ws
vector_section_line = *ws float 1*ws float 1*ws float 1*ws integer 1*ws
    integer *ws
section_kind_vector = "Velocities" / "Forces" / "Magmoms" / "Displacements
    " / "Spreads"
section_kind_scalar = "Energies" / "Charges" / "Spins"
section_kind = section_kind_vector / section_kind_scalar
section_label_vector = *ws section_kind_vector 1*ws "of Component" 1*ws
    positive_integer *ws
section_label_scalar = *ws section_kind_scalar 1*ws "of Component" 1*ws
    positive_integer *ws
float = finite_float / non_finite
finite_float = ["+" / "-"] (1*DIGIT ["." *DIGIT] / "." 1*DIGIT) [("e" / "E
    ") ["+" / "-"] 1*DIGIT]
non_finite = ["+" / "-"] ("infinity" / "nan" / "inf")
integer = 1*DIGIT
```

Table S4: The same band through each writer and reader pair among Python, C, C++, Fortran and Julia. Fortran calls `rkr_frame_add_atom_full` in the C header order. The columns are the columns of Table 12.

Route	File (Å)	Modes	Steps	$\Delta E^{\ddagger}$ (eV)	Max displacement (Å)
CON 3.1 Python to Python	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Python to C	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Python to C++	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Python to Fortran	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Python to Julia	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C to Python	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C to C	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C to C++	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C to Fortran	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C to Julia	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C++ to Python	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C++ to C	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C++ to C++	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C++ to Fortran	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 C++ to Julia	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Fortran to Python	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Fortran to C	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Fortran to C++	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Fortran to Fortran	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Fortran to Julia	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Julia to Python	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Julia to C	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Julia to C++	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Julia to Fortran	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
CON 3.1 Julia to Julia	0	56	60	-2.31×10^{-14}	3.55×10^{-15}

```
positive_integer = (%x31-39) *DIGIT
ws = SP / HTAB
NL = CRLF / LF
nl = NL
```

References

[1] R. Goswami, Two-dimensional RMSD projections for reaction path visualization and validation, MethodsX 16 (2026) 103851. doi:10.1016/j.mex.2026.103851.
URL https://linkinghub.elsevier.com/retrieve/pii/S2215016126000683

[2] B. Efron, Better bootstrap confidence intervals, Journal of the American

Table S5: Strong scaling, cold page cache. Each repeat drops the page cache before the timed call. Median, minimum and maximum of five repeats.

Path	Ranks	Median (s)	Min	Max	Frames/rank	SHA
read, shared file	1	9.904	9.438	11.857	3167380	match
read, shared file	2	5.677	5.282	5.829	1583690	match
read, shared file	4	3.220	3.105	3.367	791846	match
read, shared file	8	2.113	2.015	2.341	395923	match
read, scatter	1	9.949	9.704	10.123	3167380	match
read, scatter	2	5.594	5.473	5.694	1583690	match
read, scatter	4	3.517	3.483	3.616	791846	match
read, scatter	8	2.438	2.355	2.498	395923	match
write, MPI-IO	1	8.721	8.240	8.975	3167380	match
write, MPI-IO	2	4.288	4.217	4.385	1583690	match
write, MPI-IO	4	2.304	2.269	2.350	791846	match
write, MPI-IO	8	1.477	1.383	2.765	395923	match
write, gather	1	9.233	8.917	9.606	3167380	match
write, gather	2	5.442	5.210	5.613	1583690	match
write, gather	4	3.051	3.030	3.399	791846	match
write, gather	8	2.118	2.112	2.174	395923	match
ingest, one writer	1	33.552	31.287	34.367	123860	match
ingest, one writer	2	24.208	23.934	25.860	61930	match
ingest, one writer	4	19.760	19.195	28.385	30965	match
ingest, one writer	8	18.133	17.182	21.784	15482	match
ingest, four shards	1	33.348	32.918	39.840	123860	match
ingest, four shards	2	25.548	25.153	41.806	61930	match
ingest, four shards	4	22.145	21.005	22.502	30965	match
ingest, four shards	8	19.078	18.912	23.929	15482	match

Statistical Association 82 (397) (1987) 171–185. doi:10.1080/01621459.1987.10478410.
URL http://www.tandfonline.com/doi/abs/10.1080/01621459.1987.10478410

Table S6: Strong scaling, warm page cache, every rank count. Median, minimum and maximum of five repeats.

Path	Ranks	Median (s)	Min	Max	Frames/rank	SHA
read, shared file	1	9.088	8.999	9.324	3167380	match
read, shared file	2	4.651	4.616	4.750	1583690	match
read, shared file	4	2.591	2.487	3.103	791846	match
read, shared file	8	1.504	1.416	1.570	395923	match
read, scatter	1	9.223	9.117	9.829	3167380	match
read, scatter	2	5.013	4.939	5.212	1583690	match
read, scatter	4	2.923	2.871	2.981	791846	match
read, scatter	8	1.836	1.810	1.875	395923	match
write, MPI-IO	1	8.369	8.238	8.517	3167380	match
write, MPI-IO	2	4.060	4.043	4.083	1583690	match
write, MPI-IO	4	2.305	2.260	2.378	791846	match
write, MPI-IO	8	1.416	1.366	1.491	395923	match
write, gather	1	9.490	9.405	10.879	3167380	match
write, gather	2	5.052	5.039	5.833	1583690	match
write, gather	4	3.075	3.023	3.098	791846	match
write, gather	8	2.152	2.102	2.262	395923	match
ingest, one writer	1	31.699	30.861	36.466	123860	match
ingest, one writer	2	23.305	23.141	23.703	61930	match
ingest, one writer	4	19.322	19.215	19.764	30965	match
ingest, one writer	8	16.810	16.663	17.330	15482	match
ingest, four shards	1	32.549	32.387	32.734	123860	match
ingest, four shards	2	25.452	25.326	27.023	61930	match
ingest, four shards	4	21.259	20.199	24.906	30965	match
ingest, four shards	8	18.805	18.644	18.879	15482	match

Table S7: Weak scaling, cold page cache. Median, minimum and maximum of five repeats.

Path	Ranks	Median (s)	Min	Max	Frames/rank	SHA
read, shared file	1	1.244	1.198	1.545	400000	match
read, shared file	2	1.392	1.323	2.021	400000	match
read, shared file	4	1.668	1.613	2.712	400000	match
read, shared file	8	2.187	2.171	4.273	400000	match
read, scatter	1	1.314	1.249	1.322	400000	match
read, scatter	2	1.486	1.431	1.514	400000	match
read, scatter	4	1.824	1.794	1.855	400000	match
read, scatter	8	2.628	2.541	7.193	400000	match
write, MPI-IO	1	1.051	1.039	1.062	400000	match
write, MPI-IO	2	1.091	1.080	1.096	400000	match
write, MPI-IO	4	1.201	1.189	1.300	400000	match
write, MPI-IO	8	1.436	1.367	1.797	400000	match
write, gather	1	1.228	1.213	1.248	400000	match
write, gather	2	1.412	1.247	1.544	400000	match
write, gather	4	1.530	1.510	1.946	400000	match
write, gather	8	2.079	2.038	2.699	400000	match
ingest, one writer	1	7.900	7.869	8.444	30800	match
ingest, one writer	2	11.893	11.675	18.447	30800	match
ingest, one writer	4	20.264	19.865	23.177	30800	match
ingest, one writer	8	29.606	29.503	40.565	30800	match
ingest, four shards	1	7.760	7.589	9.905	30800	match
ingest, four shards	2	11.088	10.998	12.714	30800	match
ingest, four shards	4	18.153	17.970	20.462	30800	match
ingest, four shards	8	35.345	34.356	44.072	30800	match

Table S8: Weak scaling, warm page cache, every rank count. Median, minimum and maximum of five repeats.

Path	Ranks	Median (s)	Min	Max	Frames/rank	SHA
read, shared file	1	1.161	1.149	1.189	400000	match
read, shared file	2	1.208	1.165	1.235	400000	match
read, shared file	4	1.355	1.311	1.366	400000	match
read, shared file	8	1.581	1.573	1.792	400000	match
read, scatter	1	1.218	1.169	1.242	400000	match
read, scatter	2	1.278	1.277	1.382	400000	match
read, scatter	4	1.525	1.478	1.561	400000	match
read, scatter	8	1.962	1.949	2.264	400000	match
write, MPI-IO	1	1.037	1.032	1.088	400000	match
write, MPI-IO	2	1.085	1.075	1.103	400000	match
write, MPI-IO	4	1.199	1.190	1.227	400000	match
write, MPI-IO	8	1.382	1.365	1.446	400000	match
write, gather	1	1.211	1.164	1.238	400000	match
write, gather	2	1.317	1.309	1.322	400000	match
write, gather	4	1.572	1.507	1.613	400000	match
write, gather	8	2.161	2.097	2.296	400000	match
ingest, one writer	1	7.841	7.701	7.923	30800	match
ingest, one writer	2	11.518	11.392	11.854	30800	match
ingest, one writer	4	18.902	18.720	19.846	30800	match
ingest, one writer	8	29.594	28.943	30.364	30800	match
ingest, four shards	1	7.465	7.452	7.497	30800	match
ingest, four shards	2	10.856	10.821	10.891	30800	match
ingest, four shards	4	18.375	18.172	19.113	30800	match
ingest, four shards	8	34.899	34.807	36.533	30800	match